

16. Bölüm

Akışlar

16.1 Akış Nedir?

Akış

C++'ta **akış** (**stream**), verilerin bir kaynaktan (klavye, dosya, ağ) bir hedefe (ekran, dosya, bellek) doğru aktığı **soyut bir "kanal"** veya "**boru hattı**" olarak düşünülür. Akışlar, verinin fiziksel olarak **nerede** saklandığıyla ilgilenmez; sadece **verinin sıralı bir şekilde transfer edilmesini sağlar**.

16.2 Akış Hiyerarşisi ve Temel Sınıflar

C++ dilinde akış işlemleri `<iostream>` başlığı altında toplanan bir sınıf hiyerarşisine dayanır:

- ♦ **ios_base**: Tüm akış sınıfları için taban sınıftır. Biçimlendirme bayraklarını (**hex**, **dec**, **fixed** vb.) yönetir.
- ♦ **istream**: Giriş akışları için kullanılır (Örnek olarak klavyeden veri okuyan `std::cin`).
- ♦ **ostream**: Çıkış akışları için kullanılır (Örnek olarak konsola veri yazan `std::cout` ve standart hata çıktısı yazdığımız akış olan `std::cerr`).
- ♦ **iostream**: Hem giriş hem de çıkış yapabilen akışlardır.

16.3 Standart Akış Nesneleri

Her C++ programı başladığında dört temel akış nesnesi otomatik olarak oluşturulur ve sürekli açıktır:

Standart Akışlar	Akış Nesnesi	Sınıf	Açıklama
Standart Giriş	<code>std::cin</code> (<code>std::cinistream</code>)	<code>istream</code>	Standart giriş (Genellikle klavye): Geleneksel klavye girişini okumak için kullanılır.
Standart Çıktı	<code>std::cout</code> (<code>std::coutostream</code>)	<code>ostream</code>	Standart çıkış (Genellikle ekran). Tamponlanmıştır (buffered). Geleneksel olarak konsola çıktıyı yazmak için kullanılır.
Standart Hata	<code>std::cerr</code> (<code>std::cerrostream</code>)	<code>ostream</code>	Standart hata çıkışı. Tamponlanmamıştır. Hatalar anında basılır. Verileri bir ara belleğe kaydetmez. Bu, hataların hemen görüntülenmesi gerekir.
Standart İz Kaydı	<code>std::clog</code> (<code>std::clogostream</code>)	<code>ostream</code>	İz kaydı (log) yada bir başka deyişle günlük çıkışı. Tamponlanmıştır (buffered). Verileri bir ara belleğe kaydeder. Çünkü iz kayıtlarına hemen bakılması gerekmez.

Tablo 16.1: Standart Akış Nesneleri

16.4 Akış İşleçleri

Akışlar iki temel işleç (**operator**) ile yönetilir:

1. **Ekleme İşleci** (**<<**) (**insertion operator**): Veriyi akışa gönderir (Konsol veya dosyadan).
2. **Ayıklama İşleci** (**>>**) (**extraction operator**): Veriyi akıştan çeker (Klavye veya dosyadan). Ayıklama işleci varsayılan olarak beyaz boşlukları (**white space**) yani boşluk, tab, yeni satır gibi karakterleri atlar. Tüm satırı okumak için `std::getline()` fonksiyonu tercih edilmelidir.

16.5 Standart olmayan Akış Nesneleri

Yazdığımız programlarda sadece girdi ve çıktı işlemleri yaparsak, program çalıştığı sürece veriler var olur, program sonlandırıldığında o verileri tekrar kullanamayız. Bunun için elektrik kesildiğinde bellekteki veriler kaybolacağından harici hafıza ortamında (**external memory**) verileri saklamak gerekir.

Kısaca **dosya** (**file**), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**secondar stor-**

age). Bu ikincil ortam; Bir bilgisayar belleği (**memory**) olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı sabit disk (**hard disc**), ya da bir başka ortama/bilgisayara veri gönderen (**send**) veya alan (**receive**) modem ve benzeri bir cihaz (**device**) olabilir.

Dosya akışları (**file streams**) ile çalışmak için `<fstream>` başlığı kullanılır. Akış mantığı burada da aynıdır, sadece kaynak klavye değil diskteki bir dosyadır. Dosya ile işlem yapmak için üç akış sınıfı kullanılır; `ifstream` sınıfı dosyadan **veri okumak için**, `ofstream` sınıfı dosyaya **veri yazmak için** ve `fstream` sınıfı ise dosyaya **hem veri yazmak, hem de dosyadan veri okumak için** kullanılır.

Standart Olmayan Akışlar Kısa Sürede Kapatılmalıdır

Standart olmayan akışlar sürekli açık olamadıkları için ile açık (open**) işlemiz bittikten sonra kapatmamız (**close**) gerekir.** Çünkü bu kaynakları kullanmanın maliyeti yüksektir.

Bir akış **veri okumak veya veri yazmak için** açmak istersek `open()` yöntemi kullanılır;

```
void open (const char* filename, ios_base::openmode mode);
void open (const string& filename, ios_base::openmode mode); // C++11
```

Yönteminin ilk argümanı açılacak dosyanın adını ve konumunu belirtirken, ikinci argümanı ise dosyanın hangi modda açılacağını tanımlar. Bunlar;

İşlem Modu	Açıklama
<code>ios::in</code>	Dosya okumak için yani sadece veri girdisi (input) için kullanılır.
<code>ios::out</code>	Dosya yazmak için yani dosyaya sadece veri göndermek (output) için kullanılır.
<code>ios::binary</code>	İşlemler metin yerine ikili (binary) modda gerçekleştirilir.
<code>ios::ate</code>	Dosyanın sonunda (at the end) gidilerek hem okuma hem de yazma amacıyla dosya açılır.
<code>ios::app</code>	Gönderilen bütün veri, dosyanın sonuna (append) eklenir.
<code>ios::trunc</code>	Mevcut bir dosya varsa açmadan önce içi boşaltılır (truncate).

Tablo 16.2: Dosya Açma Modları

Dosya Açma Modları

Dosya açma modu; `ifstream` sınıfı için `ios_base::in`, `ofstream` sınıfı için `ios_base::out` ve `fstream` sınıfı için `ios_base::in|ios_base::out` **varsayılan argümandır**. Dosya açma modlarından iki veya daha fazlasını bit düzeyi VEYA (`|`) işleciyle birleştirebiliriz!

Akış nesnesi olarak açılan dosyaya aynı `std::cout` nesnesinde olduğu gibi akışa veri **ekleme işleci** (**insertion operator**) (`<<`) ile veri ekleyebilir veya `std::cin` nesnesinde olduğu gibi akıştan veri **ayıklama işleci** (**extraction operator**) (`>>`) ile veri okuyabilirsiniz. Aşağıda programın çalıştığı klasörde `veriler.txt` adlı bir metin dosyasına metin yazılıp okuma örneği verilmiştir.

```
#include <fstream> // ofstream, ifstream, open() ve close()
#include <string>
void dosyaOrnegi() {
    // Dosyaya yazma
    std::ofstream yazici("veriler.txt"); // mode=ios_base::out
    if (yazici.is_open()) {
        yazici << "Merhaba C++ Akislari!" << std::endl;
        yazici.close();
    }
    // Dosyadan okuma
    std::ifstream okuyucu("veriler.txt"); // mode=ios_base::in
    std::string satir;
    while (std::getline(okuyucu, satir)) { // Tüm satırı okumak için getline()
        std::cout << "Dosyadan okundu: " << satir << std::endl;
    }
    okuyucu.close(); //açtığımız akışı kapatmalıyız!
}
```

Örneğin programın çalıştığı klasörde `cikti.txt` adlı bir dosyayı her seferinde içeri boşaltarak sadece yazmak için akış nesnesi aşağıdaki gibi açılabilir.

```
std::ofstream outfile;
outfile.open("cikti.txt", ios::out | ios::trunc );
if(outfile.is_open()){
    os << "Merhaba İlhan!";
}
```

Benzer şekilde, aynı dosyayı hem okuma ve hem de yazma amacıyla aşağıdaki gibi açabilirsiniz;

```
std::fstream dosya;
dosya.open("cikti.txt", ios::out | ios::in );
if (!dosya.is_open()) {
    throw CustomException(afile, "Dosya Açılmadı!");
}
```

Çıktı dosyası akışında ekleme işleci (<<) yerine, write() yöntemi de kullanılabilir:

```
if(afile.is_open()){
    char metin[] = "İlhan";
    os.write(metin, 3); // Metinden 3 karakter akışa yazılıyor
}
```

Programın çalıştığı klasörde metin.txt dosyasında aşağıdaki verilerin olduğunu varsayalım;

İlhan Özkan 25 4 6 1987

Mehmet Yıldırım 15 5 24 1976

Bu durumda dosyadan veri okuma aşağıdaki şekilde yapılabilir;

```
std::ifstream is("metin.txt");
std::string adi, soyadi;
int yas, dogumAyi, dogumGunu, dogumYili;
while (is >> adi >> soyadi >> yas >> dogumAyi >> dogumGunu >> dogumYili)
cout << adi << " " << soyadi;
```

Açık Olan Dosyaların Kapatılması

Bir C++ programından çıkıldığında otomatik olarak tüm akışları temizler, ayrılmış belleği serbest bırakır ve açık tüm dosyaları kapatır. **Ancak bir programcının programı sonlandırmadan önce açık olan tüm dosyaları kapatması her zaman iyi bir uygulamadır.**

Aşağıda dosya açma ve kapatmaya bir başka örnek verilmiştir;

```
#include <fstream>
#include <iostream>
#include <string>
int main () {
    std::string adiSoyadi;
    int yas;
    char cinsiyet;
    std::cout << "Adınızı ve Soyadınızı Giriniz: ";
    std::getline(std::cin, adiSoyadi);
    std::cout << "Yaşınızı Giriniz: ";
    std::cin >> yas;
    std::cout << "cinsiyetinizi Giriniz (K-E): ";
    std::cin >> cinsiyet;

    //Verileri kalıcı olarak dosyaya yazalım:
    std::ofstream outfile; // çıkış akışı nesnesi tanımlandı.
    outfile.open("output.txt"); //dosya açıldı
    outfile << "Bu dosya Yaş ve cinsiyet Bilgilerini İçerir" << std::endl;
    outfile << "Adı:" << adiSoyadi << std::endl;
    outfile << "Yaşı:" << yas << std::endl;
    outfile << "cinsiyeti:" << cinsiyet << std::endl;
    outfile.close(); // dosya kapandı

    //Yardığımız dosyadan verileri okuyalım:
    std::string satir;
    std::ifstream infile; // giriş akışı nesnesi tanımlandı.
```

```

infile.open("output.txt"); //dosya açıldı
std::getline(infile, satir);
std::cout << "Dosyadan Okunan Satır:" << std::endl << satir << std::endl;
std::getline(infile, satir);
std::cout << "Dosyadan Okunan Bir Sonraki Satır:" << std::endl << satir << std::endl;
infile.close(); // dosya kapandı
}
/*Program Çıktısı:
Adınızı Giriniz: İlhan Özkan
Yaşınızı Giriniz: 50
Cinsiyetinizi Giriniz (K-E): E
Dosyadan Okunan Satır:
Bu dosya Yaş ve Cinsiyet Bilgilerini İçerir
Dosyadan Okunan Bir Sonraki Satır:
Adı:İlhan Özkan
*/

```

Program çalıştığında oluşan `output.txt` metin dosyasının içeriği aşağıda verilmiştir;

```

Bu dosya Yaş ve Cinsiyet Bilgilerini İçerir
Adı:İlhan Özkan
Yaşı:50
Cinsiyeti:E

```

Metin Dosyaları Gözle Okunabilir!

Buraya kadar işlediğimiz akışlar konsola yazdığımız veya klavyeden okuduğumuz gibi metin olarak çalışır. **Yani oluşan dosyalar gözle okunabilir metinlerdir.**

16.6 Akış Biçimlendirme

Verinin nasıl görüneceğini kontrol etmek için `<iomanip>` başlığındaki **manipülâtörler** (**manipulator**) kullanılır. Bu biçimlendirme aslında 5.3 başlığında detaylı verilmiştir. Aşağıda çok kullanılanları verilmiştir;

- ◊ `std::setw(n)` : Verinin çıktıya gönderilirken kaç karakter aralıkla biçimlendirileceğini yani genişliğini (**width**) belirler.
- ◊ `std::setprecision(n)` : Ondalıklı sayıların çıktıya gönderilirken kesir kısmındaki basamak sayısını yani duyarlılığını ayarlar.
- ◊ `std::fixed`/`std::scientific` : Kayan noktalı sayı biçimini belirler.
- ◊ `std::hex`, `std::dec`, `std::oct` : Çıktıya gönderilecek tam sayının sayı tabanını değiştirir.

16.7 Metin Akışları

Bazen bir metni (**string**) sanki bir giriş akışıymış gibi parçalamak veya farklı türdeki verileri bir metinde birleştirmek gerekebilir. Bunun için bellekte oluşturulan **metin akışları** (**string streams**) kullanılır ve `<sstream>` başlık dosyasında yer alır. Metin akışlarına bellek akışı (**memory stream**) da denir.

```

#include <sstream>
std::stringstream ss;
ss << "Hız:" << 120 << " km/s";
std::string rapor = ss.str(); // "Hız:120 km/s"

```

16.8 Akış Durum Kontrolleri

Bir akışın sağlıklı çalışıp çalışmadığını kontrol etmek için **bayraklar** (**flags**) kullanılır: Akış durumlarını (**stream states**) kontrol eden yöntemler aşağıda verilmiştir;

- ◊ `good()`: Her şey yolunda.


```

int sayi;
std::cout << "Bir sayi girin: ";
if (std::cin >> sayi && std::cin.good()) {
    std::cout << "Basariyla okundu: " << sayi << std::endl;
}

```
- ◊ `fail()`: Tip uyumsuzluğu gibi bir hata oluştu (Örn: int beklerken harf girilmesi).


```

int yas;
std::cout << "Yasinizi girin: ";

```

```

std::cin >> yas;
if (std::cin.fail()) {
    std::cout << "Hata: Sayı yerine gecersiz bir karakter girdiniz!" << std::endl;
    std::cin.clear(); // Hata bayragini temizler
    std::cin.ignore(1000, '\n'); // Tampondaki hatali girisi atlar
}
♦ eof(): Dosyanın veya akışın sonuna gelindi (End of File).
std::ifstream dosya("veriler.txt");
char karakter;
while (dosya.get(karakter)) {
    // Okuma yapıyor...
}
if (dosya.eof()) {
    std::cout << "Dosyanın sonuna başarıyla ulaşıldı." << std::endl;
}
♦ bad(): Ciddi bir sistem hatası (Örneğin dosyaya yazarken disk doldu).
std::ofstream cikis("yedek.db");
cikis << "Cok buyuk veri seti...";
if (cikis.bad()) {
    std::cerr << "Kritik Hata: Yazma islemi sırasında sistem hatasi olustu!" << std::endl;
}

```

16.9 İkili Akışlar

Çalışılan dosyalar, yukarıdaki gibi her zaman metin dosyası olamayabilir. Bir nesneyi bellekte olduğu gibi yani ikili olarak aynen dosyaya saklamak isteyebiliriz. Bu durumda akışları **ikili dosya** (**binary file**) yazıp okuyacak şekilde açıp işlem yapmalıyız. Bu durumda dosya açma modu olarak **ios::binary** kullanılmalıdır. Aşağıda ikili olarak oluşturulan bir dosya örneği verilmiştir.

```

#include <iostream>
#include <fstream>
struct Kisi {
    char adiSoyadi[16];
    int yas;
    char cinsiyet;
    float kilo;
};
int main() {
    Kisi kisi1={"Ilhan OZKAN",50,'E',100.0};
    Kisi kisi2={"Yagmur OZKAN",45,'K',60.0};
    int dizi[5]={1,2,3,4,5};
    std::ofstream os("kisi.bin", std::ios::binary);
    if (!os) {
        std::cerr << "Dosya Açılamadı!" << std::endl;
        return 1;
    }
    //Veriler baytlar halinde peşpeşe akışa yazılıyor.
    os.write(reinterpret_cast<char*>(&kisi1), sizeof(Kisi));
    os.write(reinterpret_cast<char*>(&kisi2), sizeof(Kisi));
    os.write(reinterpret_cast<char*>(&dizi), sizeof(dizi));
    os.close();
}

```

Programın çalışması oluşan **kisi.bin** dosyasına ilk önce iki adet kişi yapısı kaydedilmiştir. Sonrasında 5 tam sayıdan oluşan dizi kaydedilmiştir. Bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği gözle okunamaz. İkili dosyaları açan görüntüleyiciler ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında **certutil -encodehex kisi.bin kisi.txt** komutu verilebilir;

```

C:\Users\ILHANOZKAN>certutil -encodehex kisi.bin kisi.txt
Input Length = 77
Output Length = 367
CertUtil: -encodehex command completed successfully.
C:\Users\ILHANOZKAN>

```

Benzer işlem Linux ortamında `hexdump -C kisi.bin | less >> kisi.txt` komutuyla yapılabilir. Artık dönüştürülen dosyayı (`kisi.txt`) istediğimiz metin editöründe açıp okuyabiliriz.

```
0000 49 6c 68 61 6e 20 4f 5a 4b 41 4e 00 00 00 00 00  İlhan OZKAN.....
0010 32 00 00 00 00 45 00 00 00 00 00 c3 88 42 59 61 67  2...E.....BYag
0020 6d 75 72 20 4f 5a 4b 41 4e 00 00 00 00 2d 00 00  mur OZKAN....-..
0030 00 4b 00 00 00 00 00 70 42 01 00 00 00 02 00 00  .K.....pB.....
0040 00 03 00 00 00 04 00 00 00 05 00 00 00 00 00 00  .....
```

Oluşturulan aynı dosyayı ikili olarak okuyan program aşağıdaki şekilde yazılabilir. Burada yazım sırasındaki nesneleri okumak çok önemlidir. Yani dosyaya yazan kişi neyi hangi sırada yazdığından okumayı da aynı programcı yapmalıdır;

```
#include <iostream>
#include <fstream>
struct Kisi {
    char adiSoyadi[16];
    int yas;
    char cinsiyet;
    float kilo;
};
int main() {
    Kisi kisi1, kisi2;
    int dizi[5];
    std::ifstream is("kisi.bin", std::ios::binary);
    if (!is) {
        std::cerr << "Dosya Açılamadı!" << std::endl;
        return 1;
    }
    is.read(reinterpret_cast<char*>(&kisi1), sizeof(Kisi));
    std::cout << "Okunan Kisi Yapısı:" << std::endl << "Kişi Adı:" << kisi1.adiSoyadi <<
    << std::endl << "Kişi Yaşı:" << kisi1.yas << std::endl << "Kişi Cinsiyeti:" <<
    << kisi1.cinsiyet << std::endl << "Kişi Kilo:" << kisi1.kilo << std::endl;
    is.read(reinterpret_cast<char*>(&kisi2), sizeof(Kisi));
    std::cout << "Okunan Kisi Yapısı:" << std::endl << "Kişi Adı:" << kisi2.adiSoyadi <<
    << std::endl << "Kişi Yaşı:" << kisi2.yas << std::endl << "Kişi Cinsiyeti:" <<
    << kisi2.cinsiyet << std::endl << "Kişi Kilo:" << kisi2.kilo << std::endl;
    is.read(reinterpret_cast<char*>(&dizi), sizeof(dizi));
    std::cout << "Okunan Dizi:" << dizi[0] << "," << dizi[1] << "," << dizi[2] << "," << dizi[3]
    << "," << dizi[4] << std::endl;
    is.close();
}
/* Program Çalıştığında:
Kişi Kilo:100
Okunan Kisi Yapısı:
Kişi Adı:Yagmur OZKAN
Kişi Yaşı:45
Kişi Cinsiyeti:K
Kişi Kilo:60
Okunan Dizi:1,2,3,4,5
*/
```

Eğer sadece diziyi okumak isteseydik dosya konum göstericisini okuyacağımız yere konumlandırmak gerekir. Bu durumda okuyucu program aşağıdaki gibi olurdu:

```
#include <iostream>
#include <fstream>
struct Kisi {
    char adiSoyadi[16];
    int yas;
    char cinsiyet;
    float kilo;
};
int main() {
    Kisi kisi1, kisi2;
```

```

int dizi[5];

std::ifstream is("kisi.bin", std::ios::binary);
if (!is) {
    std::cerr << "Dosya Açılmadı!" << std::endl;
    return 1;
}
is.seekg(2*sizeof(Kisi),std::ios::beg); // iki adet Kisi yapısı kadar ilerlendi!
// dizi iki kişi kaydı sonrasında kaydedilmişti.
is.read(reinterpret_cast<char*>(&dizi), sizeof(dizi));
std::cout << "Okunan Dizi:" << dizi[0] << "," << dizi[1] << "," << dizi[2] << "," << dizi[3]
    << "," << dizi[4] << std::endl;
is.close();
}
/* Program Çalıştığında:
Okunan Dizi:1,2,3,4,5
*/

```

Hem `istream` hem de `ostream` sınıfındaki akışlarda ikili olarak açılan dosya için dosya konum göstericisini yeniden konumlandırmak için yöntemler sağlar. Bunlar; Okuma durumunda imlecin konumunu (indisini) öğrenmek için `seekg()` kullanılır. Yazma durumunda ise yeniden konumlandırma için `seekp()` yöntemi kullanılır. Bu yöntemlerde konumlandırmanın yönü, bir akışın **başlangıcına göre** konumlandırma `ios::beg` ile yapılır (varsayılan). Konumlandırma yönü, bir akıştaki geçerli konuma göre yapılması için `ios::cur` veya bir akışın sonuna göre konumlandırma yapılması için `ios::end` bayrakları kullanılır;

```

#include <iostream>
#include <fstream>
int main() {
    // 1. Dosyayı hem okuma hem yazma modunda açıyoruz
    std::fstream dosya("ornek.txt", std::ios::out | std::ios::in | std::ios::trunc);
    if (!dosya) {
        std::cerr << "Dosya açılmadı!" << std::endl;
        return 1;
    }
    // Dosyaya ilk veriyi yazalım
    dosya << "Merhaba Dünya";
    // 2. "Dunya" kelimesini degistirmek istiyoruz.
    // "Merhaba " (8 karakter) sonrasına gitmemiz lazım.
    // seekp(8) -> Dosyanın başından itibaren 8. bayta git demektir.
    dosya.seekp(8); // Varsayılan olarak dosyanın başından: ios::beg
    // 3. Yeni veriyi tam o noktadan itibaren yazıyoruz
    dosya << "C++";
    // Sonucu kontrol etmek için imleci başa alıp okuyalım
    dosya.seekg(0); // Varsayılan olarak dosyanın başına: ios::beg
    std::string icerik;
    std::getline(dosya, icerik);
    std::cout << "Dosyanın son hali: " << icerik << std::endl;
    // Çıktı: Merhaba C++ya (C++ 3 harf olduğu için sadece 'Dun' kısmını ezdi)
    dosya.close();
}

```

Eğer bir dosyayı hem okuma hem yazma (`ios::in|ios::out`) modunda açtıysanız, `seekg()` ve `seekp()` imleçleri çoğu sistemde ortaktır. Yani birini hareket ettirdiğinizde diğeri de hareket eder. Bu yüzden **okuma işleminden yazma işlemine geçerken imleç konumunu her zaman kontrol etmelisiniz**.

`seekg()` yöntemi ile bir dosyanın yerine gidiyoruz ama "Şu an dosyanın tam olarak neresindeyim?" diye sormak istersen `tellg()` ve `tellp()` fonksiyonları kullanılır. Okuma imlecinin mevcut konumunu (indisini) `tellg()` döndürür. Yazma imlecinin mevcut konumunu ise `tellp()` döndürür. `tellg()` yöntemi, okuma imlecinin yerini söyler. En yaygın kullanım alanı, **imleci sona gönderip konumunu alarak dosya boyutunu bulmaktır**.

```

#include <iostream>
#include <fstream>
int main() {
    std::ifstream dosya("veriler.txt", std::ios::binary);

```



```

if (!dosya) {
    std::cerr << "Dosya acilamadi!" << std::endl;
    return 1;
}
// 1. İmleci dosyanın en sonuna götür
dosya.seekg(0, std::ios::end);
// 2. tellg() ile şu anki konumu (yani toplam bayt sayısını) al
std::streampos boyut = dosya.tellg();
std::cout << "Dosyanın toplam boyutu: " << boyut << " bayt." << std::endl;
dosya.close();
}

```

`tellp()` yöntemi, yazma imlecinin o an hangi baytta olduğunu söyler. Büyük bir dosyaya veri eklerken nerede kaldığımızı kaydetmek için idealdir.

```

#include <iostream>
#include <fstream>
int main() {
    std::ofstream cikis("kayit.txt");
    cikis << "Satir 1" << std::endl; // "Satir 1\n" yazıldı
    // Şu an kaçınıcı bayttayız?
    std::streampos konum1 = cikis.tellp();
    std::cout << "İlk yazımdan sonraki konum: " << konum1 << std::endl;
    cikis << "İkinci Uzun Satir";
    std::streampos konum2 = cikis.tellp();
    std::cout << "İkinci yazımdan sonraki konum: " << konum2 << std::endl;
    cikis.close();
}

```

İmleç Konumu Olarak std::streampos Kullanımı

1. **Büyük Dosyalar:** standart bir `int`, 2 GB'den büyük dosyaların adresini tutamaz. `streampos` ise çok daha büyük dosya boyutlarını destekler.
2. **Platform Bağımsızlık:** Farklı işletim sistemlerinde dosya konumu işaretleme biçimleri değişebilir; `streampos` bu karmaşıklığı C++ standartlarında çözer.

16.10 std::cerr ve std::clog Nesnelerini Dosyaya Yönlendirme

Hataları izlemek için kullanılan `std::cerr` nesnesi genellikle konsola hata çıkışı vermek için kullanılsa da bir dosyaya da yönlendirilebilir. Bu, bir programda hataları günlüğe kaydetmeniz gerektiğinde yararlı olabilir.

```

#include <iostream>
#include <fstream>
int main() {
    std::ofstream errorLog("hata.txt");
    std::cerr.rdbuf(errorLog.rdbuf());
    std::cerr << "Hata mesajları bu dosyaya yazılacak!" << std::endl;
    errorLog.close();

    std::ofstream logLog("log.txt");
    std::clog.rdbuf(logLog.rdbuf());
    std::clog << "İz Kaydı (log) mesajları bu dosyaya yazılacak!" << std::endl;
    logLog.close();
}

```

Neden Akış Kullanmalıyız?

- ◊ **Tip Güvenliği:** C dilindeki `printf()` fonksiyonundaki gibi biçim belirleyici karakterler (`%d,%f`) gerektirmez; veri tipinin kendisi anlar.
- ◊ **Genişletilebilirlik:** Kendi yazdığınız sınıflar için ayıklama (`<<`) ve ekleme (`>>`) işlemlerini aşırı yükleyerek (**overloading**), sınıflarınızı standart akışlarla uyumlu hale getirebilirsiniz.
- ◊ **Soyutlama:** Verinin diskten mi, internetten mi yoksa bellekten mi geldiğiyle ilgilenmeden aynı kod yapısını kullanmanıza olanak tanır.

16.11 Düşün ve Çöz

- ◇ **Düşün:** C++'da standart akışları (`std::cin`, `std::cout`, `std::cerr`, `std::clog`) karşılaştırın. `std::cerr` ile `std::clog` arasındaki tamponlama farkı ne anlama gelir ve hata ayıklamada önemi nedir?
- ◇ **Çöz:** Bir metin dosyasına öğrenci notlarını ('isim', 'numara', 'not') yazıp okuyan bir program yazın. `std::ofstream` ve `std::ifstream` kullanımını, dosya açma modlarını (`ios::app`, `ios::trunc`) açıklayın.
- ◇ **Çöz:** `std::stringstream` sınıfını kullanarak bir CSV satırını parçalara ayıran (parse eden) bir fonksiyon yazın. Bu sınıfın metin dönüşümlerinde nasıl kullanışlı olduğunu örnekleyin.
- ◇ **Çöz:** İkili (**binary**) modda dosya okuma/yazma ile metin modundaki farkı açıklayın. Bir `struct` nesnesini ikili dosyaya yazıp geri okuyan bir program tasarlayın.
- ◇ **Düşün:** Dosya akışlarında hata kontrolü nasıl yapılır? `is_open()`, `good()`, `eof()`, `fail()` ve `bad()` yöntemlerini açıklayın. Sağlam bir dosya okuma döngüsü nasıl yazılmalıdır?